CocoConnect

A Digital Ecosystem for Coconut Estate Management

System Requirements Document

Requirements refinement, feasibility review and database design

Client: Coconut Growers Association of Sri Lanka · Group 38

Prepared by: S.N. Vidanage (2023/IS/108)

Contents

Contents 2
1. Introduction 3
1.1 Purpose 3
1.2 Baseline 3
1.3 System summary 3
2. Refined Requirements by Module 3
3. Technical Feasibility Review 3
4. Consistency Review 4
5. Hardware and Software Requirements 4
5.1 Client devices 4
5.2 Server and services 4
5.3 Development environment 5
6. Non-Functional Requirements 5
7. Database Design 5
7.1 Entity Relationship diagram 5
7.2 Entities, attributes and primary keys 6
7.3 Relationships 7
7.4 Design decisions 8
8. Requirements to Database Traceability 8

1. Introduction

1.1 Purpose

This document sets out the system requirements for CocoConnect after the requirements were refined with stakeholder feedback. It reviews the technical feasibility of each module, checks the requirements for consistency, records the hardware and software the system needs, and describes the database design (entities, attributes, keys and relationships) that supports the required functions.

1.2 Baseline

The baseline is the Software Requirements Specification, Version 3 (SRS v3), which organises the requirements into modules with use case diagrams. Requirement IDs (FR-xx, NFR-xx, BR-xx) in this document refer to SRS v3.

1.3 System summary

CocoConnect is a cross-platform system (Android, iOS and web) that brings coconut product trading, workforce hiring, estate management and harvest recording into one platform, with ratings, notifications and administration. It is built with Flutter, Node.js with Express.js, and PostgreSQL, communicating through a REST API.

2. Refined Requirements by Module

The table summarises each module after refinement, the requirements it covers, and the main changes made from stakeholder feedback.

Module	SRS v3 requirements	Refinement from stakeholder feedback
User Management	FR-01 to FR-07	Six roles defined (Estate Owner, Estate Manager, Buyer, Seller, Worker/Service Provider, Administrator); profile picture and password reset added; role-based permissions made explicit
Product Marketplace	FR-08 to FR-16	Selling method per nut or per kilogram; DC coconut, curry coconut and husk as categories; search and filter by category, location and price; product images added
Workforce Marketplace	FR-17 to FR-23	Seven service categories confirmed; charging per tree or per day; job request status tracked
Estate and Harvest Management	FR-24 to FR-31	Estate statistics and harvest reports added; harvest history kept for analysis
Rating and Notification	FR-32 to FR-41	Two-way ratings (owner–worker, buyer–seller) to address trust issues raised by the client; notifications for job requests and seller contact
Administration	FR-42 to FR-47	User verification, listing management, complaint review, platform reports and suspension

3. Technical Feasibility Review

Each module was checked against the chosen stack (Flutter, Node.js/Express, PostgreSQL) and the team's skills and time.

Module	Feasibility	Reasoning	Technical approach
User Management	**High**	Standard registration and login; well supported by the stack	Hashed passwords (bcrypt), JWT tokens, role checks on every protected API route
Product Marketplace	**High**	Listing, search and filter are standard database operations	PostgreSQL tables for products and orders; filtered REST endpoints
Workforce Marketplace	**High**	Profiles and request status follow the same pattern as the marketplace	Service profiles and hiring requests with a status field
Estate Management	**High**	Simple data entry and display	Estate table linked to its owner and manager
Harvest Management	**Medium**	Offline recording needs local storage and later synchronisation	Sync_Status on each harvest record; upload when the device is online

Module	Feasibility	Reasoning	Technical approach
Location features	**Medium**	Needs a map provider and distance calculation	Store coordinates; calculate distance on the server; free map service to avoid billing
Rating and Notification	**High**	Ratings are simple records; in-app notifications can be read from the database	Rating and notification tables; push notifications later
Administration	**High**	CRUD screens over existing tables	Admin-only API routes and a web portal
Online payment	**Low**	Needs a payment gateway, merchant registration and extra security	Deferred: record payment method and status only

4. Consistency Review

SRS v3 and the database design were checked against each other and against the stakeholder requirements. The findings and their resolutions are below.

#	Finding	Resolution	Status
1	The requirement IDs in the use case description tables (e.g. 'FR-06 Product Listing', 'FR-09 Workforce Profile') do not match the numbered requirements in the same sections (FR-06 is role permissions; FR-09 is edit listings).	Use the numbered FR-01 to FR-47 list as the reference and update the description tables to match.	**Resolved**
2	Module section numbering jumps from 3.5 to 3.8.	Renumber Administration as 3.6.	**Resolved**
3	FR-02 says users log in with an email address, but stakeholders mainly use phones, and the ER diagram stores User_Phone.	Allow login with a phone number; keep email optional.	**Needs decision**
4	Multilingual support is listed as a future enhancement, but stakeholders asked for English, Sinhala and Tamil, and the ER diagram stores User_Lang_Pref.	Treat three-language support as a core requirement.	**Needs decision**
5	Payment and Inventory appear as entities in the ER diagram, while online payment and inventory management are listed as future enhancements.	Keep the entities: record payment method, amount and status, and quantity available. Online payment processing stays a future feature.	**Resolved**
6	Equipment rental is a future enhancement in the SRS, but Equipment (with Daily_Rate) is in the ER diagram and tractor services are a current service category.	Include equipment in service profiles now; full rental booking later.	**Resolved**
7	Location-based search is required (FR-11 filter by location) but GPS-based services and maps are listed as future items.	Store estate and user locations now so location filtering works; GPS tracking stays future.	**Needs decision**
8	The SRS lists both Worker and Service Provider; the ER diagram keeps them as separate subtypes.	Keep both: workers offer labour (Service_Profile), providers own equipment.	**Resolved**

5. Hardware and Software Requirements

5.1 Client devices

Item	Requirement
Mobile	Android 9.0 or above; iOS 14 or above
Memory	Minimum 2 GB RAM
Web	Modern browser: Google Chrome, Microsoft Edge or Mozilla Firefox
Network	Internet connection for most features; harvest recording also offline
Location	Device GPS for location features

5.2 Server and services

Component	Requirement
Application server	Node.js runtime with Express.js, exposing a REST API
Database	Cloud-hosted PostgreSQL server

Component	Requirement
Communication	REST over HTTPS with JSON data
Future services	Push notifications (Firebase Cloud Messaging), payment gateway

5.3 Development environment

Tool	Purpose
Flutter SDK and Dart	Mobile and web frontend
Node.js and npm	Backend development
PostgreSQL	Database
Android Studio and emulator	Running and testing the mobile app
Visual Studio Code	Code editing
Git and GitHub	Version control
Figma, draw.io	Wireframes, use case and ER diagrams

6. Non-Functional Requirements

Category	Requirements (SRS v3)	How the design supports it
Performance	NFR-01 to NFR-03: responses within 3 seconds; many users at once; efficient search	Indexed primary and foreign keys; filtering done in the database
Security	NFR-04 to NFR-07: encrypted passwords, authenticated access, role-based access, protected personal data	Password hashing, token-based login, role checks, Verif_Status on User
Reliability	NFR-08, NFR-09: no data loss; consistent records after failures	Database transactions for orders and payments; foreign keys
Availability	NFR-10, NFR-11: available 24 hours except planned maintenance	Cloud-hosted database and server
Usability	NFR-12 to NFR-14: intuitive, consistent navigation, clear errors	Role-based dashboards; User_Lang_Pref for language
Scalability	NFR-15, NFR-16: new products and modules without major change	Product_Category table lets new products be added as data
Maintainability	NFR-17, NFR-18: modular, documented code	Separate modules for frontend screens and backend routes

7. Database Design

7.1 Entity Relationship diagram

The ER diagram below shows the database entities, their attributes (primary keys underlined), and the relationships between them.

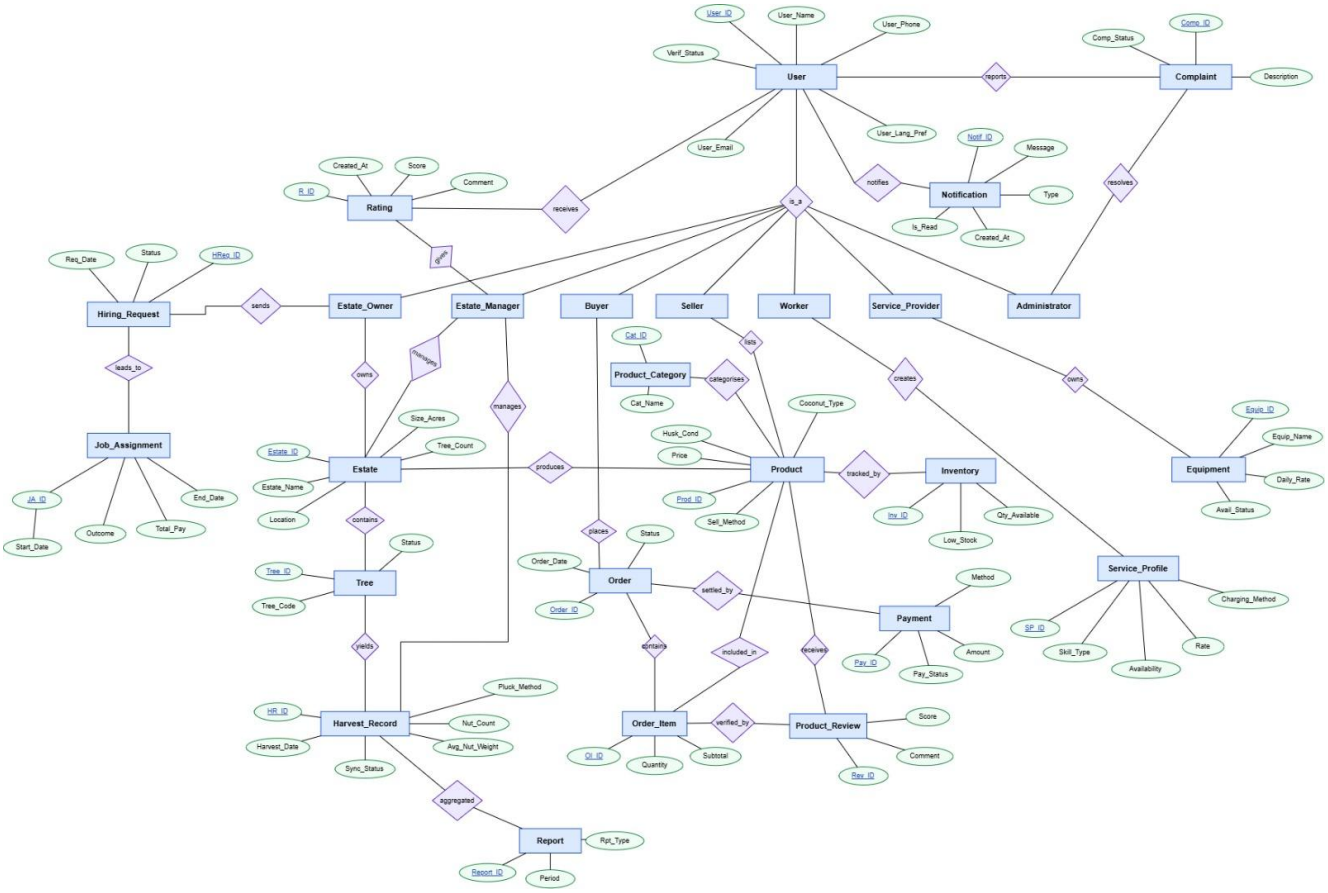


Figure 1: CocoConnect Entity Relationship diagram

7.2 Entities, attributes and primary keys

Entity	Primary key	Attributes	Purpose
User	User_ID	User_Name, User_Phone, User_Email, User_Lang_Pref, Verif_Status	Superclass of every role (is_a)
Estate_Owner / Estate_Manager / Buyer / Seller / Worker / Service_Provider / Administrator	User_ID (inherited)	Inherited from User	Subtypes of User
Estate	Estate_ID	Estate_Name, Location, Size_Acres, Tree_Count	The estate that grows the coconuts
Tree	Tree_ID	Tree_Code, Status	Individual coconut trees on an estate
Harvest_Record	HR_ID	Harvest_Date, Pluck_Method, Nut_Count, Avg_Nut_Weight, Sync_Status	One harvest from a tree; Sync_Status supports offline recording
Report	Report_ID	Rpt_Type, Period	Harvest reports built from harvest records
Product_Category	Cat_ID	Cat_Name	Groups products (coconut, husk, future products)
Product	Prod_ID	Coconut_Type, Husk_Cond, Price, Sell_Method	A product an estate produces and a seller lists
Inventory	Inv_ID	Qty_Available, Low_Stock	Stock level of a product
Order	Order_ID	Order_Date, Status	A buyer's purchase
Order_Item	OI_ID	Quantity, Subtotal	One product line inside an order
Payment	Pay_ID	Method, Amount, Pay_Status	How an order was paid

Entity	Primary key	Attributes	Purpose
Product_Review	Rev_ID	Score, Comment	Review of a product, verified by a purchased order item
Rating	R_ID	Score, Comment, Created_At	Rating of a user
Hiring_Request	HReq_ID	Req_Date, Status	An estate owner's request to hire
Job_Assignment	JA_ID	Start_Date, End_Date, Outcome, Total_Pay	The job that results from an accepted hiring request
Service_Profile	SP_ID	Skill_Type, Availability, Rate, Charging_Method	A worker's advertised service
Equipment	Equip_ID	Equip_Name, Daily_Rate, Avail_Status	Machinery a service provider rents out
Notification	Notif_ID	Message, Type, Is_Read, Created_At	Alerts sent to users
Complaint	Comp_ID	Description, Comp_Status	A user's report, resolved by an administrator

7.3 Relationships

Entity	Relationship	Related entity	Meaning
User	is_a	Estate_Owner, Estate_Manager, Buyer, Seller, Worker, Service_Provider, Administrator	Each user account has a role
Estate_Owner	owns	Estate	An owner registers and owns estates
Estate_Manager	manages	Estate	A manager runs an assigned estate
Estate	contains	Tree	Trees belong to an estate
Tree	yields	Harvest_Record	Harvests are recorded per tree
Estate_Manager	manages	Harvest_Record	Managers record harvests
Harvest_Record	aggregated	Report	Reports summarise harvest records
Estate	produces	Product	Products come from an estate
Seller	lists	Product	Sellers publish products
Product_Category	categorises	Product	Each product has a category
Product	tracked_by	Inventory	Stock level per product
Buyer	places	Order	Buyers create orders
Order	contains	Order_Item	An order has one or more lines
Product	included_in	Order_Item	Each line refers to a product
Order	settled_by	Payment	An order is paid
Product	receives	Product_Review	Products are reviewed
Order_Item	verified_by	Product_Review	Only purchased items can be reviewed
Estate_Owner	sends	Hiring_Request	Owners send hiring requests
Hiring_Request	leads_to	Job_Assignment	An accepted request becomes a job
Worker	creates	Service_Profile	Workers advertise services
Service_Provider	owns	Equipment	Providers own machinery
User	gives / receives	Rating	Users rate each other
User	notifies	Notification	Users receive notifications
User	reports	Complaint	Users report problems
Administrator	resolves	Complaint	Admins handle complaints

7.4 Design decisions

- A single User entity holds shared details, and each role is a subtype (is_a). One login system serves all roles, and permissions follow the role.
- Order and Order_Item are separate, so one order can contain several products and each line keeps its own quantity and subtotal.
- Product_Review is verified by an Order_Item, enforcing the business rule that only completed purchases can be reviewed (BR-07).
- Hiring_Request leads to Job_Assignment, so a job exists only after a request is accepted (BR-05, BR-06).
- Harvest_Record has Sync_Status to support offline recording, and Avg_Nut_Weight to record the quality indicator stakeholders use.
- Product_Category lets new agricultural products be added without changing the schema (NFR-15).
- User_Lang_Pref stores each user's language for the three-language interface.

8. Requirements to Database Traceability

Requirement area	SRS v3	Entities that support it
Registration, login, roles	FR-01 to FR-07	User and its role subtypes
Product listing and search	FR-08 to FR-16	Product, Product_Category, Inventory, Estate
Buying	FR-15, BR-03	Order, Order_Item, Payment
Workforce hiring	FR-17 to FR-23	Service_Profile, Hiring_Request, Job_Assignment, Equipment
Estate and harvest	FR-24 to FR-31	Estate, Tree, Harvest_Record, Report
Ratings and reviews	FR-32 to FR-37	Rating, Product_Review
Notifications	FR-38 to FR-41	Notification
Administration	FR-42 to FR-47	Administrator, Complaint, User (Verif_Status)